Mini Project

House Price Prediction using Machine Learning
Regression Models and Performance Analysis using Python

Submitted By:	**Submitted To:**
Name: Chirag Manav	Ms. Swati Mittal
UID: 25MCA20008	
Class: MCA	
Section: 6/B	

University Institute of Computing
Chandigarh University, Gharuan, Mohali

Index

1. Abstract **3**

2. Problem Statement and Objectives **3**

3. Dataset **3**

4. Methodology **3**

5. Results **3**

6. Output Screenshots **5**

7. Conclusion and Future Work **11**

8. Complete Source Code **12**

main.py 12

src/generate_dataset.py 13

src/preprocessing.py 14

src/train.py 14

src/visualize.py 15

src/predict.py 16

1. Abstract

This project builds an end-to-end machine learning system that predicts residential property prices for eight Indian cities (Mumbai, Delhi, Bengaluru, Pune, Hyderabad, Chandigarh, Shimla, Jaipur) from 14 property attributes such as area, bedrooms, age, floor, distance from the city centre and amenities. Five regression models are compared with 5-fold cross-validation and the best, Gradient Boosting, is tuned with GridSearchCV. The final model reaches R2 = 0.959 on unseen data.

2. Problem Statement and Objectives

Property prices depend on many interacting factors, so manual valuation is slow and inconsistent. The objectives are to (a) explore and clean the data, (b) engineer useful features, (c) compare several models, (d) tune and evaluate the best one, and (e) save it for predicting the price of new houses.

3. Dataset

The dataset has 3,000 listings and 15 columns (14 features plus the target **price_lakhs**). **Note:** the data is synthetically generated using realistic pricing rules (city rate per sq ft, area, age, distance, amenities, property type, furnishing), with about 3% missing values and a few injected outliers. Scores therefore reflect this simulated market, not real-world accuracy. A real dataset with the same columns can be dropped in without changing the code.

Feature	Type	Description
city, property_type, furnishing	Categorical	Location, Apartment/House/Villa, furnishing level
area_sqft, bedrooms, bathrooms, balconies	Numeric	Size and room counts
age_years, floor, total_floors, lift	Numeric	Building age and floor details
parking, distance_city_center_km, amenities_score	Numeric	Convenience and location quality
price_lakhs	Target	Price in INR Lakhs (1 Lakh = 100,000)

4. Methodology

• **Cleaning:** per-city IQR outlier removal on price per sq ft (11 rows removed); median/mode imputation for missing values.

• **Feature engineering:** floor_ratio, area_per_bedroom, bath_bed_ratio.

• **Preprocessing:** StandardScaler for numeric and One-Hot Encoding for categorical columns inside a scikit-learn Pipeline, so no test information leaks into training.

• **Split:** 80% train (2,391) / 20% test (598).

• **Models:** Linear, Ridge, Decision Tree, Random Forest, Gradient Boosting.

• **Tuning:** GridSearchCV over learning rate, depth, number of trees and subsample (32 settings x 5 folds).

• **Metrics:** MAE, RMSE, R2, MAPE, residual analysis, learning curve and permutation importance.

5. Results

Model	CV R2	Test R2	MAE (Lakhs)	RMSE (Lakhs)	MAPE %
Tuned Gradient Boosting	0.957	0.959	7.37	11.20	8.98
Gradient Boosting	0.941	0.949	8.61	12.50	10.63

Model	CV R2	Test R2	MAE (Lakhs)	RMSE (Lakhs)	MAPE %
Random Forest	0.926	0.923	10.19	15.30	12.43
Ridge Regression	0.876	0.883	12.15	18.85	18.01
Linear Regression	0.876	0.883	12.16	18.86	18.05
Decision Tree	0.854	0.825	14.33	23.07	17.53

The tuned model's CV R2 (0.957) is the grid-search best score; the other CV values are 5-fold means on the training set.

Key findings: ensemble methods clearly beat linear models, showing non-linear relationships between features and price. City and area dominate the predictions, followed by distance from the centre, property age and property type. Floor, furnishing and amenities have a smaller effect.

Sample predictions from the saved model:

Property	Area	Predicted price
3BHK Apartment, Bengaluru	1250 sq ft	Rs 99.22 Lakhs
2BHK Apartment, Pune	950 sq ft	Rs 51.05 Lakhs
4BHK Villa, Shimla	2400 sq ft	Rs 169.36 Lakhs
2BHK Apartment, Mumbai	800 sq ft	Rs 143.56 Lakhs

6. Output Screenshots

```
house_price_prediction -- python main.py

$ python main.py

======================================================================
 [1/5] DATA OVERVIEW
======================================================================
Raw dataset shape      : (3000, 15)
After outlier removal  : (2989, 15)  (11 rows removed)
Train / Test split     : 2391 / 598

First 5 rows:
         city       type    sqft  bed  bath  balc   age  floor  tot_fl  parking      furnish  lift  dist_km  amenity  price_L
0  Chandigarh      Villa  1551.0    3     2     2  29.0      1       3        0    Furnished     0      5.3      7.7    86.00
1        Pune      Villa   863.0    1     1     2   NaN      2       7        1  Unfurnished     1      5.6      3.1    65.86
2      Jaipur      Villa   380.0    1     1     0  17.0      1       5        1    Furnished     1     26.4      6.3    11.15
3  Chandigarh  Apartment   479.0    1     1     1   1.0      2      21        1    Furnished     1      0.7      6.1    30.35
4      Mumbai  Apartment  1690.0    4     4     3   3.0      3      24        1    Furnished     1      1.0      8.0   400.12

Missing values per column:
age_years                  90
distance_city_center_km    60
amenities_score           120

Price summary (Rs Lakhs):
count    3000.00
mean       85.17
std        58.38
min        10.92
25%        47.99
50%        69.51
75%       102.14
max       547.63
```

Fig 1 - Data overview (terminal output)

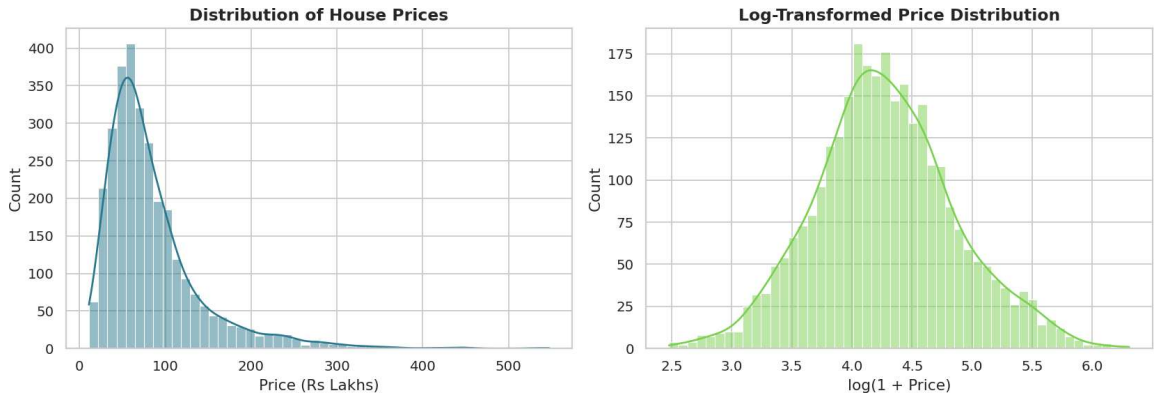


Fig 2 - Price distribution and log transform

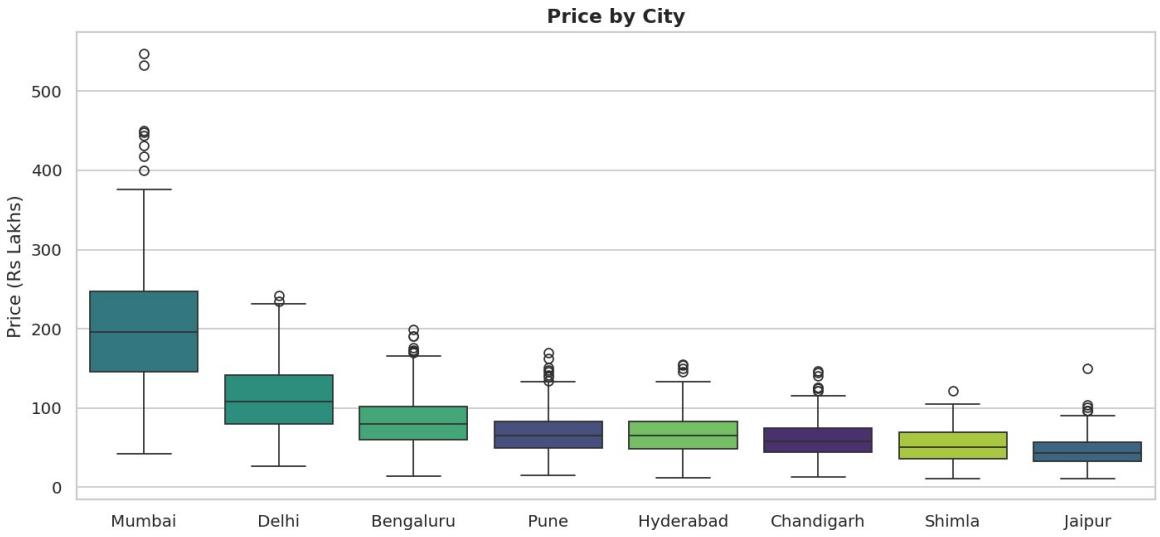


Fig 3 - Price by city



Fig 4 - Area vs price

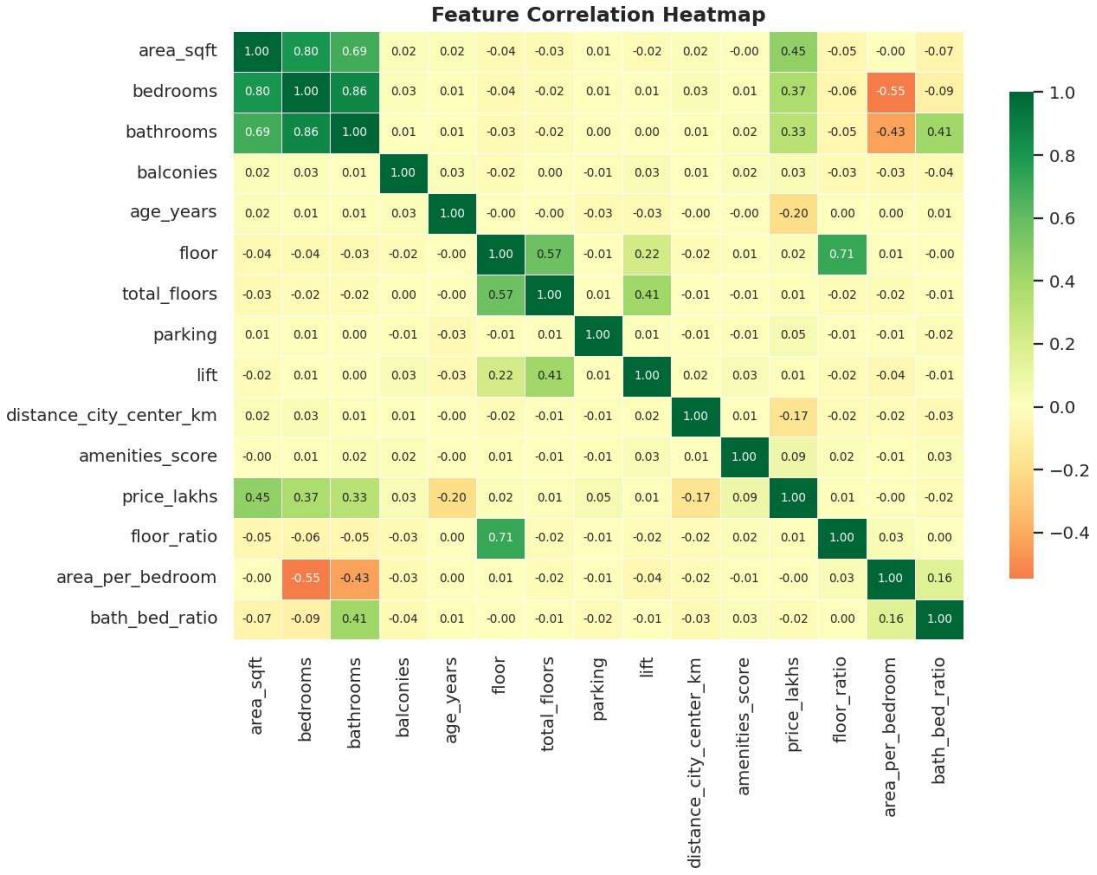


Fig 5 - Correlation heatmap

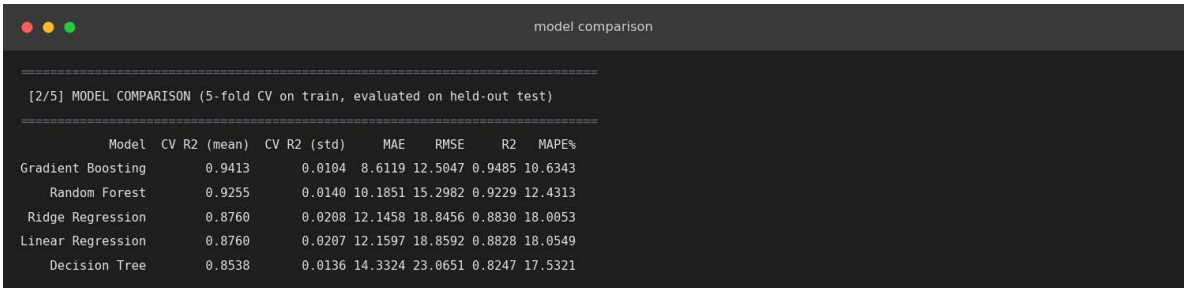


Fig 6 - Model comparison (terminal output)

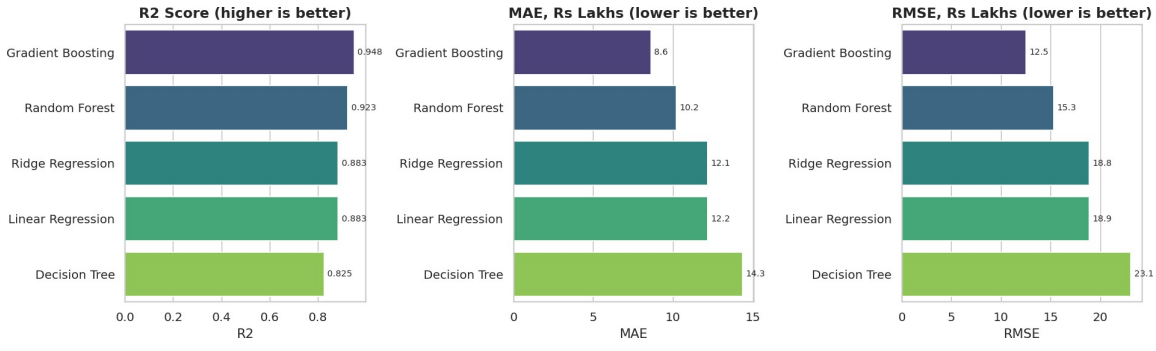


Fig 7 - Model comparison chart

hyperparameter tuning

```
================================================================================
 [3/5] HYPERPARAMETER TUNING - Gradient Boosting (GridSearchCV, 5-fold)
================================================================================
Best params   : {'model__learning_rate': 0.05, 'model__max_depth': 4, 'model__n_estimators': 400, 'model__subsample': 0.8}
Best CV R2    : 0.9568

Final TEST-SET performance (tuned model):
  MAE   : 7.3690
  RMSE  : 11.2041
  R2    : 0.9586
  MAPE% : 8.9769
```

Fig 8 - Hyperparameter tuning (terminal output)

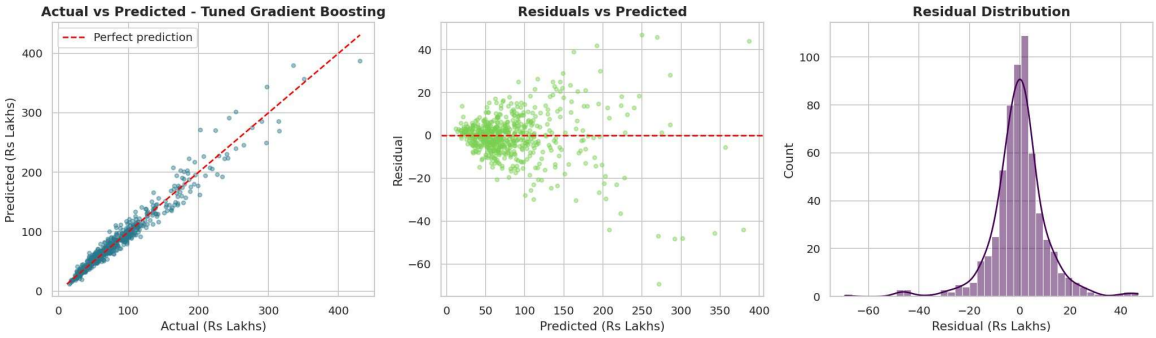


Fig 9 - Actual vs predicted and residuals

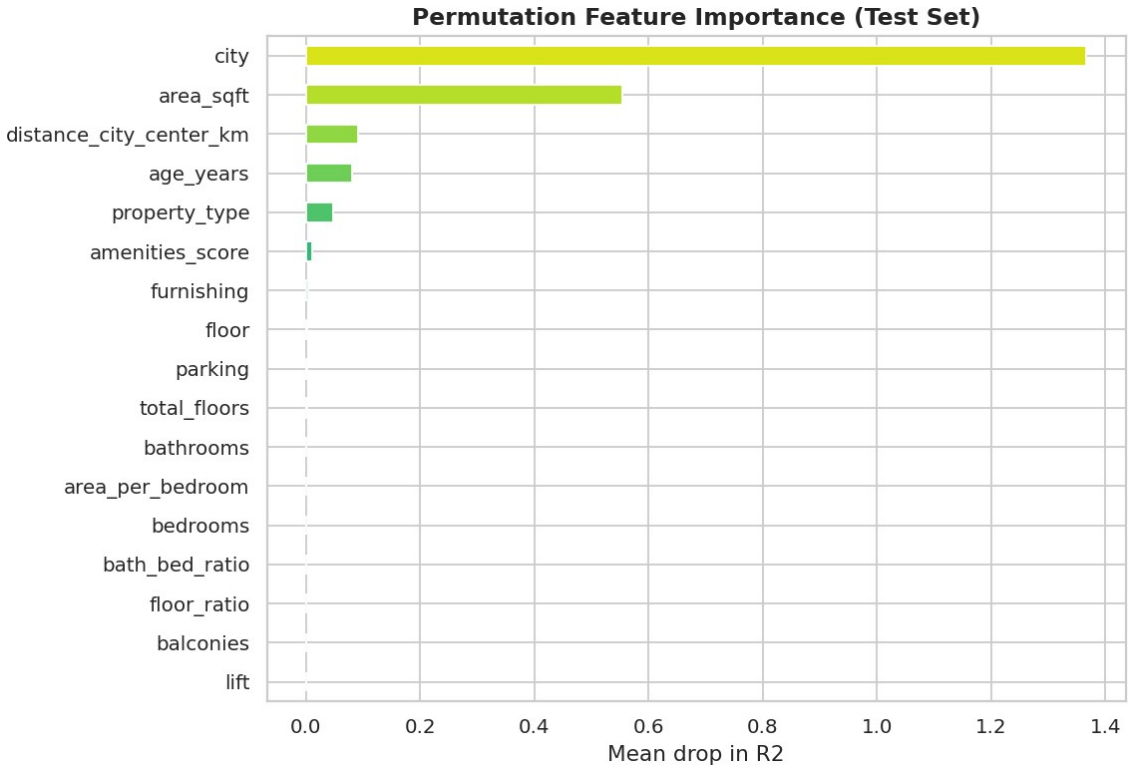


Fig 10 - Permutation feature importance

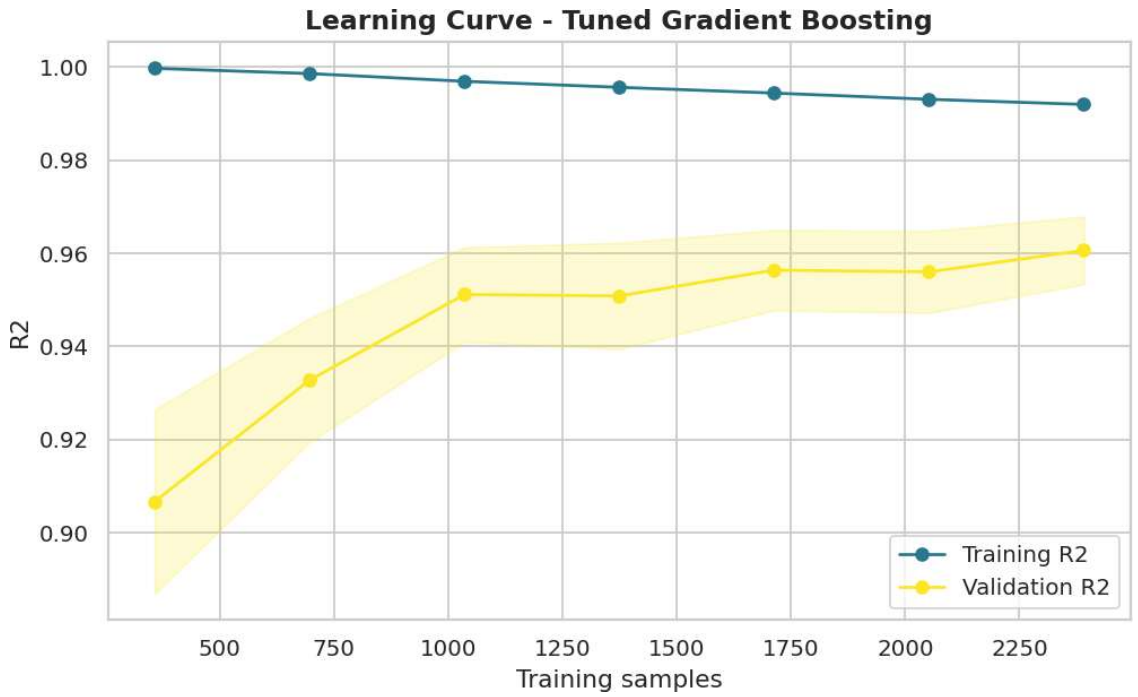


Fig 11 - Learning curve

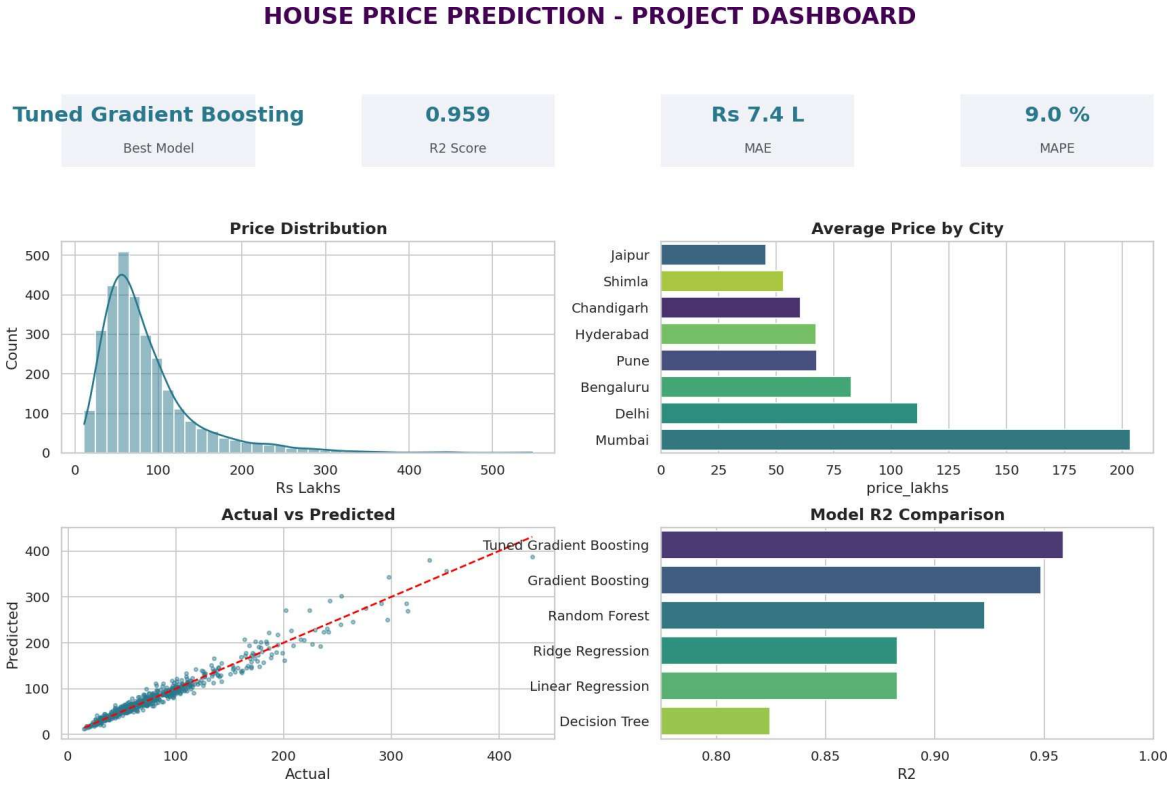


Fig 12 - Project dashboard

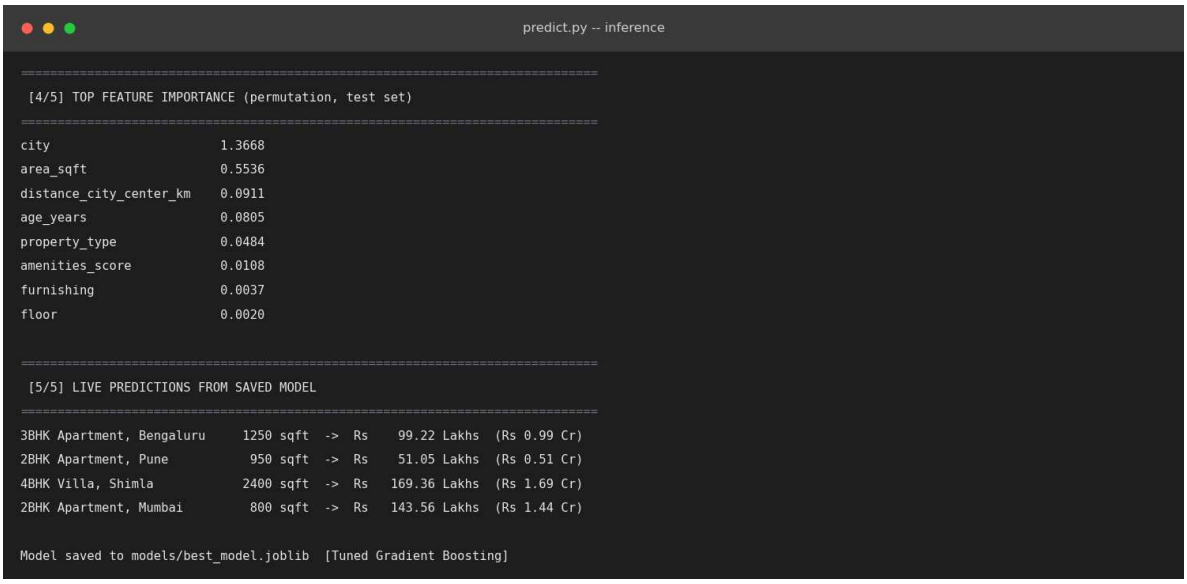


Fig 13 - Predictions from saved model (terminal output)

7.Conclusion and Future Work

The tuned Gradient Boosting model predicts prices with about 9% average error and explains 95.9% of the variance on unseen data. The learning curve shows training and validation scores converging, so the model is not badly overfitted. Future work: use real listing data, add XGBoost/LightGBM, explain predictions with SHAP, and deploy through a Streamlit or FastAPI web app.

1. Complete Source Code

Run with: `pip install pandas numpy scikit-learn matplotlib seaborn joblib`, place the files as shown (main.py at the top, the rest inside src/), then `python main.py`.

main.py

```
"""House Price Prediction - end-to-end pipeline.

Run:  python main.py
"""
import io
import sys
from contextlib import redirect_stdout
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent / "src"))

import joblib
import pandas as pd

import visualize as viz
from generate_dataset import generate
from predict import predict_price
from preprocessing import TARGET
from train import compare_models, metrics, prepare, tune_best

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 30)

for d in ("data", "models", "outputs/screenshots"):
    Path(d).mkdir(parents=True, exist_ok=True)
if not Path("data/housing_data.csv").exists():
    generate().to_csv("data/housing_data.csv", index=False)


def section(title: str) -> str:
    return f"{'=' * 78}\n {title}\n{'=' * 78}"
```

```
# ---------------------------------------------------------------------------------------- 1. DATA
buf = io.StringIO()
with redirect_stdout(buf):
    print("$ python main.py\n")
    raw, clean, (X_tr, X_te, y_tr, y_te) = prepare()
    print(section("[1/5]  DATA  OVERVIEW"))
    print(f"Raw dataset shape      : {raw.shape}")
    print(f"After outlier removal  : ({len(clean)}, {raw.shape[1]})  ({len(raw) - len(clean)} rows removed)")
    print(f"Train / Test split     : {len(X_tr)} / {len(X_te)}\n")
    print("First 5 rows:")
    print(raw.head().rename(columns={"property_type": "type", "distance_city_center_km": "dist_km", "amenities_score":
"amenity", "total_floors": "tot_fl", "furnishing": "furnish", "balconies": "balc", "bathrooms": "bath", "bedrooms":
"bed", "area_sqft": "sqft", "price_lakhs": "price_L", "age_years": "age"}).to_string())
    print("\nMissing values per column:")
    print(raw.isna().sum()[raw.isna().sum() > 0].to_string())
    print("\nPrice summary (Rs Lakhs):")
    print(raw[TARGET].describe().round(2).to_string())
txt1 = buf.getvalue(); print(txt1)
viz.terminal_shot(txt1, "01_data_overview.png", "house_price_prediction -- python main.py")

viz.eda_plots(clean)

# ---------------------------------------------------------------------------------------- 2. MODELS
buf = io.StringIO()
with redirect_stdout(buf):
    print(section("[2/5] MODEL COMPARISON (5-fold CV on train, evaluated on held-out test)"))
    res, fitted = compare_models(X_tr, X_te, y_tr, y_te)
    print(res.round(4).to_string(index=False))
txt2 = buf.getvalue(); print(txt2)
viz.terminal_shot(txt2, "06a_model_results_terminal.png", "model comparison")
viz.model_comparison(res)

# ---------------------------------------------------------------------------------------- 3. TUNING
buf = io.StringIO()
with redirect_stdout(buf):
    print(section("[3/5] HYPERPARAMETER TUNING - Gradient Boosting (GridSearchCV, 5-fold)"))
    gs = tune_best(X_tr, y_tr)
    print(f"Best params    : {gs.best_params_}")
    print(f"Best CV R2     : {gs.best_score_:.4f}")
    best = gs.best_estimator_
    pred = best.predict(X_te)
    m = metrics(y_te, pred)
    print("\nFinal TEST-SET performance (tuned model):")
    for k, v in m.items():
        print(f"  {k:<6}: {v:,.4f}")

txt3 = buf.getvalue(); print(txt3)
viz.terminal_shot(txt3, "07a_tuning_terminal.png", "hyperparameter tuning")

# pick final model: tuned GB vs best untuned
final_name, final_model, final_pred = "Tuned Gradient Boosting", best, pred
best_untuned = res.iloc[0]
if best_untuned["R2"] > m["R2"]:
    final_name = best_untuned["Model"]; final_model = fitted[final_name]
    final_pred = final_model.predict(X_te); m = metrics(y_te, final_pred)

viz.prediction_plots(y_te.values, final_pred, final_name)
imp = viz.feature_importance(final_model, X_te, y_te)
viz.learning_curve_plot(final_model, pd.concat([X_tr, X_te]), pd.concat([y_tr, y_te]))
final_row = pd.DataFrame([{"Model": final_name, **m}])
viz.dashboard(clean, pd.concat([final_row, res]).reset_index(drop=True), y_te.values,
              final_pred, final_name)

# ---------------------------------------------------------------------------------------- 4. EXPLAIN + 5. PREDICT
joblib.dump(final_model, "models/best_model.joblib")
buf = io.StringIO()
with redirect_stdout(buf):
    print(section("[4/5] TOP FEATURE IMPORTANCE (permutation, test set)"))
    print(imp.head(8).round(4).to_string())
    print("\n" + section("[5/5] LIVE PREDICTIONS FROM SAVED MODEL"))
    houses = [
        ("3BHK Apartment, Bengaluru", dict(city="Bengaluru", property_type="Apartment", area_sqft=1250,
            bedrooms=3, bathrooms=2, balconies=2, age_years=4, floor=7, total_floors=15, parking=1,
            furnishing="Semi-Furnished", lift=1, distance_city_center_km=8.5, amenities_score=7.5)),
        ("2BHK Apartment, Pune", dict(city="Pune", property_type="Apartment", area_sqft=950,
            bedrooms=2, bathrooms=2, balconies=1, age_years=8, floor=3, total_floors=10, parking=1,
            furnishing="Unfurnished", lift=1, distance_city_center_km=12, amenities_score=6)),
        ("4BHK Villa, Shimla", dict(city="Shimla", property_type="Villa", area_sqft=2400,
            bedrooms=4, bathrooms=4, balconies=2, age_years=3, floor=0, total_floors=2, parking=2,
            furnishing="Furnished", lift=0, distance_city_center_km=5, amenities_score=8.5)),
        ("2BHK Apartment, Mumbai", dict(city="Mumbai", property_type="Apartment", area_sqft=800,
            bedrooms=2, bathrooms=2, balconies=1, age_years=10, floor=12, total_floors=20, parking=1,
            furnishing="Semi-Furnished", lift=1, distance_city_center_km=10, amenities_score=7)),
    ]
    for label, h in houses:
        p = predict_price(h)
        print(f"{label:<28} {h['area_sqft']:>5} sqft  ->  Rs {p:>8,.2f} Lakhs  (Rs {p / 100:,.2f} Cr)")
    print(f"\nModel saved to models/best_model.joblib  [{final_name}]")
txt4 = buf.getvalue(); print(txt4)
viz.terminal_shot(txt4, "11_predictions_terminal.png", "predict.py -- inference")
```

```
print("\nAll screenshots saved to outputs/screenshots/")
```

src/generate_dataset.py

```
"""Generate a realistic synthetic Indian housing dataset (prices in INR Lakhs).

Replace data/housing_data.csv with a real dataset (e.g. from Kaggle) that has the
same columns and the rest of the pipeline works unchanged.
"""
import numpy as np
import pandas as pd

RNG = np.random.default_rng(42)
CITY_RATE = {  # base price per sqft (INR)
    "Mumbai": 21000, "Delhi": 11500, "Bengaluru": 8500, "Pune": 7000,
    "Hyderabad": 6800, "Chandigarh": 6200, "Shimla": 5600, "Jaipur": 4800,
}


def generate(n: int = 3000) -> pd.DataFrame:
    city = RNG.choice(list(CITY_RATE), n, p=[.10, .14, .17, .14, .14, .10, .05, .16])
    area = np.clip(RNG.normal(1150, 380, n), 380, 3500).round()
    bedrooms = np.clip(np.round(area / 480 + RNG.normal(0, .5, n)), 1, 5).astype(int)
    bathrooms = np.clip(bedrooms - RNG.integers(0, 2, n), 1, 5)
    balcony = RNG.integers(0, 4, n)
    age = RNG.integers(0, 31, n)
    total_floors = RNG.integers(2, 26, n)
    floor = np.array([RNG.integers(0, t + 1) for t in total_floors])
    parking = RNG.integers(0, 3, n)
    furnishing = RNG.choice(["Unfurnished", "Semi-Furnished", "Furnished"], n, p=[.35, .40, .25])
    lift = ((total_floors > 4) & (RNG.random(n) > .08)).astype(int)
    dist = np.clip(RNG.gamma(2.2, 4.0, n), .5, 40).round(1)
    amenities = np.clip(RNG.normal(6, 2, n), 1, 10).round(1)
    prop_type = RNG.choice(["Apartment", "Independent House", "Villa"], n, p=[.72, .20, .08])

    rate = np.array([CITY_RATE[c] for c in city], dtype=float)
    mult = (1 - .012 * age) * (1 - .018 * np.minimum(dist, 25)) * (1 + .025 * (amenities - 6))
    mult *= np.where(prop_type == "Villa", 1.35, np.where(prop_type == "Independent House", 1.15, 1.0))
    mult *= np.select([furnishing == "Furnished", furnishing == "Semi-Furnished"], [1.08, 1.04], 1.0)
    mult *= 1 + .006 * np.minimum(floor, 15) * lift + .03 * parking + .01 * bathrooms + .008 * balcony
    price = area * rate * mult * RNG.lognormal(0, .08, n) / 1e5  # in lakhs

    df = pd.DataFrame(dict(
        city=city, property_type=prop_type, area_sqft=area, bedrooms=bedrooms,
```

```
        bathrooms=bathrooms, balconies=balcony, age_years=age, floor=floor,
        total_floors=total_floors, parking=parking, furnishing=furnishing, lift=lift,
        distance_city_center_km=dist, amenities_score=amenities, price_lakhs=price.round(2)))
    # realism: missing values + a few outliers
    for col, frac in [("age_years", .03), ("amenities_score", .04), ("distance_city_center_km", .02)]:
        df.loc[RNG.choice(n, int(n * frac), replace=False), col] = np.nan
    idx = RNG.choice(n, 12, replace=False)
    df.loc[idx, "price_lakhs"] *= RNG.uniform(1.8, 2.6, 12)
    return df


if __name__== "__main__":
    data = generate()
    data.to_csv("data/housing_data.csv", index=False)
    print(f"Saved {data.shape[0]} rows x {data.shape[1]} columns -> data/housing_data.csv")
```

src/preprocessing.py

```
"""Cleaning, feature engineering and sklearn preprocessing pipeline."""
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

TARGET = "price_lakhs"
CATEGORICAL = ["city", "property_type", "furnishing"]
NUMERIC = ["area_sqft", "bedrooms", "bathrooms", "balconies", "age_years", "floor",
           "total_floors", "parking", "lift", "distance_city_center_km", "amenities_score",
           "floor_ratio", "area_per_bedroom", "bath_bed_ratio"]


def remove_outliers(df: pd.DataFrame) -> pd.DataFrame:
    """Drop rows whose price-per-sqft is beyond 3 IQR of the city's price-per-sqft band."""
    pps = df[TARGET] * 1e5 / df["area_sqft"]
    q1 = pps.groupby(df["city"]).transform(lambda s: s.quantile(.25))
    q3 = pps.groupby(df["city"]).transform(lambda s: s.quantile(.75))
    iqr = q3 - q1
    return df[(pps >= q1 - 3 * iqr) & (pps <= q3 + 3 * iqr)].reset_index(drop=True)


def add_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df["floor_ratio"] = df["floor"] / df["total_floors"].clip(lower=1)
    df["area_per_bedroom"] = df["area_sqft"] / df["bedrooms"].clip(lower=1)
    df["bath_bed_ratio"] = df["bathrooms"] / df["bedrooms"].clip(lower=1)
    return df


def build_preprocessor() -> ColumnTransformer:
    num = Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler())])
    cat = Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                    ("ohe", OneHotEncoder(handle_unknown="ignore", sparse_output=False))])
    return ColumnTransformer([("num", num, NUMERIC), ("cat", cat, CATEGORICAL)])
```

src/train.py

```
"""Train, cross-validate, tune and compare regression models."""
import numpy as np
import pandas as pd
from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV, KFold, cross_val_score, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeRegressor

from preprocessing import TARGET, add_features, build_preprocessor, remove_outliers


def metrics(y_true, y_pred) -> dict:
    return {"MAE": mean_absolute_error(y_true, y_pred),
            "RMSE": np.sqrt(mean_squared_error(y_true, y_pred)),
            "R2": r2_score(y_true, y_pred),
            "MAPE%": np.mean(np.abs((y_true - y_pred) / y_true)) * 100}


def prepare(path="data/housing_data.csv"):
    raw = pd.read_csv(path)
    clean = add_features(remove_outliers(raw))
    X, y = clean.drop(columns=TARGET), clean[TARGET]
    return raw, clean, train_test_split(X, y, test_size=.2, random_state=42)


def compare_models(X_tr, X_te, y_tr, y_te):
    models = {
        "Linear Regression": LinearRegression(),
        "Ridge Regression": Ridge(alpha=1.0),
```

```
        "Decision Tree": DecisionTreeRegressor(max_depth=10, random_state=42),
        "Random Forest": RandomForestRegressor(n_estimators=200, n_jobs=-1, random_state=42),
        "Gradient Boosting": GradientBoostingRegressor(random_state=42),
    }
    kf = KFold(5, shuffle=True, random_state=42)
    rows, fitted = [], {}
    for name, est in models.items():
        pipe = Pipeline([("prep", build_preprocessor()), ("model", est)])
        cv = cross_val_score(pipe, X_tr, y_tr, cv=kf, scoring="r2", n_jobs=-1)
        pipe.fit(X_tr, y_tr)
        m = metrics(y_te, pipe.predict(X_te))
        rows.append({"Model": name, "CV R2 (mean)": cv.mean(), "CV R2 (std)": cv.std(), **m})
        fitted[name] = pipe
    return pd.DataFrame(rows).sort_values("R2", ascending=False).reset_index(drop=True), fitted


def tune_best(X_tr, y_tr):
    pipe = Pipeline([("prep", build_preprocessor()),
                     ("model", GradientBoostingRegressor(random_state=42))])
    grid = {"model__n_estimators": [200, 400], "model__learning_rate": [.05, .1],
            "model__max_depth": [3, 4], "model__subsample": [.8, 1.0]}
    gs = GridSearchCV(pipe, grid, cv=5, scoring="r2", n_jobs=-1)
    gs.fit(X_tr, y_tr)
    return gs
```

src/visualize.py

```
"""All charts + 'terminal screenshot' renderer."""
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.inspection import permutation_importance

OUT = "outputs/screenshots/"
sns.set_theme(style="whitegrid", palette="viridis")
plt.rcParams.update({"figure.dpi": 130, "axes.titleweight": "bold", "axes.titlesize": 13})


def terminal_shot(text: str, filename: str, title="Terminal"):
    """Render console output as a terminal-window screenshot."""
    lines = text.rstrip().split("\n")
    line_h, bar_h = .235, .45
    h = bar_h + .2 + line_h * len(lines)
    fig = plt.figure(figsize=(11.5, h), facecolor="#1e1e1e")
    ax = fig.add_axes([0, 0, 1, 1]); ax.axis("off"); ax.set_xlim(0, 1); ax.set_ylim(0, 1)
    ax.add_patch(plt.Rectangle((0, 1 - bar_h / h), 1, bar_h / h, color="#3a3a3a"))
    for i, c in enumerate(["#ff5f56", "#ffbd2e", "#27c93f"]):
        ax.scatter([.02 + i * .018], [1 - bar_h / 2 / h], s=45, color=c)
    ax.text(.5, 1 - bar_h / 2 / h, title, color="#cfcfcf", ha="center", va="center", fontsize=9)
    for i, ln in enumerate(lines):
        y = 1 - (bar_h + .1 + line_h * (i + .5)) / h
        color = "#e6e6e6"
        if ln.startswith("$"): color = "#7ee787"
        elif ln.startswith(("=", "-")): color = "#6e7681"
        elif ln.startswith("["): color = "#79c0ff"
        ax.text(.015, y, ln, color=color, family="monospace", fontsize=8.6, va="center")
    fig.savefig(OUT + filename, facecolor=fig.get_facecolor()); plt.close(fig)


def eda_plots(df: pd.DataFrame):
    fig, ax = plt.subplots(1, 2, figsize=(13, 4.6))
    sns.histplot(df["price_lakhs"], bins=50, kde=True, ax=ax[0], color="#2a788e")
    ax[0].set(title="Distribution of House Prices", xlabel="Price (Rs Lakhs)")
    sns.histplot(np.log1p(df["price_lakhs"]), bins=50, kde=True, ax=ax[1], color="#7ad151")
    ax[1].set(title="Log-Transformed Price Distribution", xlabel="log(1 + Price)")
    plt.tight_layout(); plt.savefig(OUT + "02_price_distribution.png"); plt.close()

    order = df.groupby("city")["price_lakhs"].median().sort_values(ascending=False).index
    plt.figure(figsize=(11, 5.2))
    sns.boxplot(data=df, x="city", y="price_lakhs", order=order, hue="city", legend=False, palette="viridis")
    plt.title("Price by City"); plt.ylabel("Price (Rs Lakhs)"); plt.xlabel("")
    plt.tight_layout(); plt.savefig(OUT + "03_price_by_city.png"); plt.close()

    plt.figure(figsize=(10, 5.5))
    sns.scatterplot(data=df.sample(1500, random_state=1), x="area_sqft", y="price_lakhs",
                    hue="city", alpha=.65, s=30)
    plt.title("Area vs Price (coloured by city)"); plt.xlabel("Area (sq ft)"); plt.ylabel("Price (Rs Lakhs)")
    plt.legend(bbox_to_anchor=(1.01, 1), loc="upper left", fontsize=8)
    plt.tight_layout(); plt.savefig(OUT + "04_area_vs_price.png"); plt.close()

    num = df.select_dtypes("number")
    plt.figure(figsize=(10.5, 8))
    sns.heatmap(num.corr(), annot=True, fmt=".2f", cmap="RdYlGn", center=0, annot_kws={"size": 7.5},
                linewidths=.4, cbar_kws={"shrink": .8})
    plt.title("Feature Correlation Heatmap"); plt.tight_layout()
    plt.savefig(OUT + "05_correlation_heatmap.png"); plt.close()
```

```
def model_comparison(res: pd.DataFrame):
    fig, ax = plt.subplots(1, 3, figsize=(15, 4.6))
    titles = {"R2": "R2 Score (higher is better)", "MAE": "MAE, Rs Lakhs (lower is better)",
              "RMSE": "RMSE, Rs Lakhs (lower is better)"}
    for a, col in zip(ax, ["R2", "MAE", "RMSE"]):
        d = res.sort_values(col, ascending=(col != "R2"))
        sns.barplot(data=d, y="Model", x=col, hue="Model", legend=False, palette="viridis", ax=a)
        for c in a.containers:
            a.bar_label(c, fmt="%.3f" if col == "R2" else "%.1f", padding=3, fontsize=8)
        a.set_title(titles[col]); a.set_ylabel("")
    plt.tight_layout(); plt.savefig(OUT + "06_model_comparison.png"); plt.close()


def prediction_plots(y_true, y_pred, name):
    resid = y_true - y_pred
    fig, ax = plt.subplots(1, 3, figsize=(16, 4.8))
    ax[0].scatter(y_true, y_pred, alpha=.45, s=14, color="#2a788e")
    lim = [min(y_true.min(), y_pred.min()), max(y_true.max(), y_pred.max())]
    ax[0].plot(lim, lim, "r--", lw=1.5, label="Perfect prediction"); ax[0].legend()
    ax[0].set(title=f"Actual vs Predicted - {name}", xlabel="Actual (Rs Lakhs)", ylabel="Predicted (Rs Lakhs)")
    ax[1].scatter(y_pred, resid, alpha=.45, s=14, color="#7ad151"); ax[1].axhline(0, color="r", ls="--")
    ax[1].set(title="Residuals vs Predicted", xlabel="Predicted (Rs Lakhs)", ylabel="Residual")
    sns.histplot(resid, bins=40, kde=True, ax=ax[2], color="#440154")
    ax[2].set(title="Residual Distribution", xlabel="Residual (Rs Lakhs)")
    plt.tight_layout(); plt.savefig(OUT + "07_actual_vs_predicted.png"); plt.close()


def feature_importance(pipe, X_te, y_te):
    pi = permutation_importance(pipe, X_te, y_te, n_repeats=8, random_state=42, n_jobs=-1)
    s = pd.Series(pi.importances_mean, index=X_te.columns).sort_values()
    plt.figure(figsize=(9, 6.2))
    s.plot.barh(color=sns.color_palette("viridis", len(s)))
    plt.title("Permutation Feature Importance (Test Set)"); plt.xlabel("Mean drop in R2")
    plt.tight_layout(); plt.savefig(OUT + "08_feature_importance.png"); plt.close()
    return s.sort_values(ascending=False)


def learning_curve_plot(pipe, X, y):
    from sklearn.model_selection import learning_curve
    sizes, tr, va = learning_curve(pipe, X, y, cv=5, scoring="r2", n_jobs=-1,
                                   train_sizes=np.linspace(.15, 1, 7))
    plt.figure(figsize=(8, 5))
    plt.plot(sizes, tr.mean(1), "o-", label="Training R2", color="#2a788e")
    plt.plot(sizes, va.mean(1), "o-", label="Validation R2", color="#fde725")
    plt.fill_between(sizes, va.mean(1) - va.std(1), va.mean(1) + va.std(1), alpha=.2, color="#fde725")
    plt.title("Learning Curve - Tuned Gradient Boosting"); plt.xlabel("Training samples"); plt.ylabel("R2")
    plt.legend(); plt.tight_layout(); plt.savefig(OUT + "09_learning_curve.png"); plt.close()


def dashboard(df, res, y_true, y_pred, best):
    """One-page summary dashboard screenshot."""
    fig = plt.figure(figsize=(16, 9.5), facecolor="white")
    gs = fig.add_gridspec(3, 4, height_ratios=[.55, 1.6, 1.6], hspace=.45, wspace=.55)
    fig.suptitle("HOUSE PRICE PREDICTION - PROJECT DASHBOARD", fontsize=19, fontweight="bold",
                 y=.985, color="#440154")
    kpis = [("Best Model", best), ("R2 Score", f"{res.iloc[0]['R2']:.3f}"),
            ("MAE", f"Rs {res.iloc[0]['MAE']:.1f} L"), ("MAPE", f"{res.iloc[0]['MAPE%']:.1f} %")]
    for i, (k, v) in enumerate(kpis):
        a = fig.add_subplot(gs[0, i]); a.axis("off")
        a.add_patch(plt.Rectangle((0, 0), 1, 1, color="#f1f3f8", transform=a.transAxes))
        a.text(.5, .62, v, ha="center", fontsize=17, fontweight="bold", color="#2a788e", transform=a.transAxes)
        a.text(.5, .2, k, ha="center", fontsize=10.5, color="#555", transform=a.transAxes)
    a = fig.add_subplot(gs[1, :2]); sns.histplot(df["price_lakhs"], bins=40, kde=True, ax=a, color="#2a788e")
    a.set_title("Price Distribution"); a.set_xlabel("Rs Lakhs")
    a = fig.add_subplot(gs[1, 2:]); order = df.groupby("city")["price_lakhs"].mean().sort_values().index
    sns.barplot(data=df, y="city", x="price_lakhs", order=order, hue="city", legend=False,
                palette="viridis", errorbar=None, ax=a)
    a.set_title("Average Price by City"); a.set_ylabel("")
    a = fig.add_subplot(gs[2, :2]); a.scatter(y_true, y_pred, s=9, alpha=.45, color="#2a788e")
    lim = [y_true.min(), y_true.max()]; a.plot(lim, lim, "r--")
    a.set_title("Actual vs Predicted"); a.set_xlabel("Actual"); a.set_ylabel("Predicted")
    a = fig.add_subplot(gs[2, 2:])
    sns.barplot(data=res, y="Model", x="R2", hue="Model", legend=False, palette="viridis", ax=a)
    a.set_xlim(max(0, res["R2"].min() - .05), 1); a.set_title("Model R2 Comparison"); a.set_ylabel("")
    plt.savefig(OUT + "10_project_dashboard.png", bbox_inches="tight"); plt.close()
```

src/predict.py

```
"""Load the saved model and predict the price of new houses."""
import joblib
import pandas as pd

from preprocessing import add_features

MODEL_PATH = "models/best_model.joblib"


def predict_price(house: dict, model_path: str = MODEL_PATH) -> float:
    """Return the predicted price in INR Lakhs for a single house description."""
```

```
    model = joblib.load(model_path)
    return  float(model.predict(add_features(pd.DataFrame([house])))[0])


if __name__== "__main__":
    sample = dict(city="Bengaluru", property_type="Apartment", area_sqft=1250, bedrooms=3,
                  bathrooms=2, balconies=2, age_years=4, floor=7, total_floors=15, parking=1,
                  furnishing="Semi-Furnished", lift=1, distance_city_center_km=8.5,
                  amenities_score=7.5)
    print(f"Predicted price: Rs {predict_price(sample):,.2f} Lakhs")
```